

intro:

So to begin this assignment ill just briefly say what ill be writing which ill first start by talking about the 3 different parts and how they progressively get better at image classification. there will be hand built ones, CNN and transfer learning.

part1:

So for the first part we start with how i built a simple AI. So what i did really was make a single neuron class that had some weights and biases which were put in and then let it get the weighted sum with my python loop. After that it would take z and could take it through any of the different activation functions which i had put in as sigmoid, relu, leaky relu and tanh. that was pretty much all for part 1A by making my first single neuron.

For part 1b i had to bring in NumPy. When it came to bringing it in all i really had to do was rewrite the single neuron with the use of np.dot which pretty much does the same thing as part 1A but it executes it 1 time rather than the entire loop i had to do for 1A. In here i also was able to generalise more by using denselayer which stored the weights in a matrix and just computed the entire layer. Which in the end makes 128 outputs vectors in 1 operation.

Going onwards to 1C which is where Pytorch comes to play. Basically what i did was i replaced the whole hand built part of computing and so on and just changed it into the pytorcehs neural network module nn.Linear and then built 3 different layers. pytorch definetly made it a lot easier by giving me code like loss.backwards, optim.adam and dataloader which i had to write all by myself before in the end the test accuracy came to about around 97% but the training curve did start to show that it was going past the test accuracy since it started just flattening which i believe is where i think overfitting started happening.

Now just to briefly go through 1D i never had to explicitly do this since i started off with it but i just moved the whole epoch and training and so on to my gpu with my code model.to(device) which did make my training time faster but i have yet to test how slow it wouldve been without it. But in general i think it really only matters in big/larger models since those most probably are what needs a lot of matrixes and so on to learn.

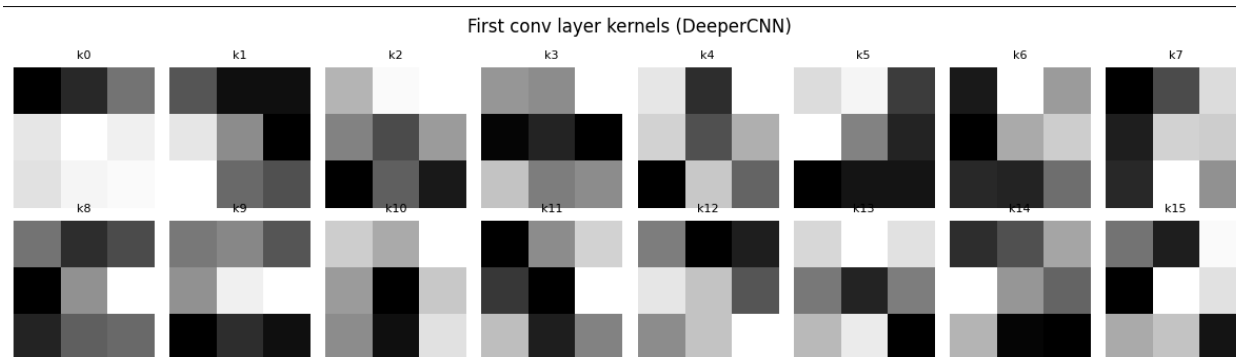
Part2:

So here is where i started on part 2, it first began by me writing a runlogger class so that my runs could always be reproducible. Basically for each run it would create a timestamped folder which writes the config which has it hyperparameters being persistent before training even starts. Then it appends every epoch to the history so that id be able to check even when its training. Then it saves both the periodic checkpoints and its best checkpoint wherever the loss is getting better. I think this is definitely really good to have since ud be able to just always know what hyperparameters you used if you need to know that or to have something to go against overfitting.

Now when it came to the data augmentation part it basically started by me applying ar randomrotation, randomaffine and a gaussiannoise. Basically the idea behind it all is that whilst

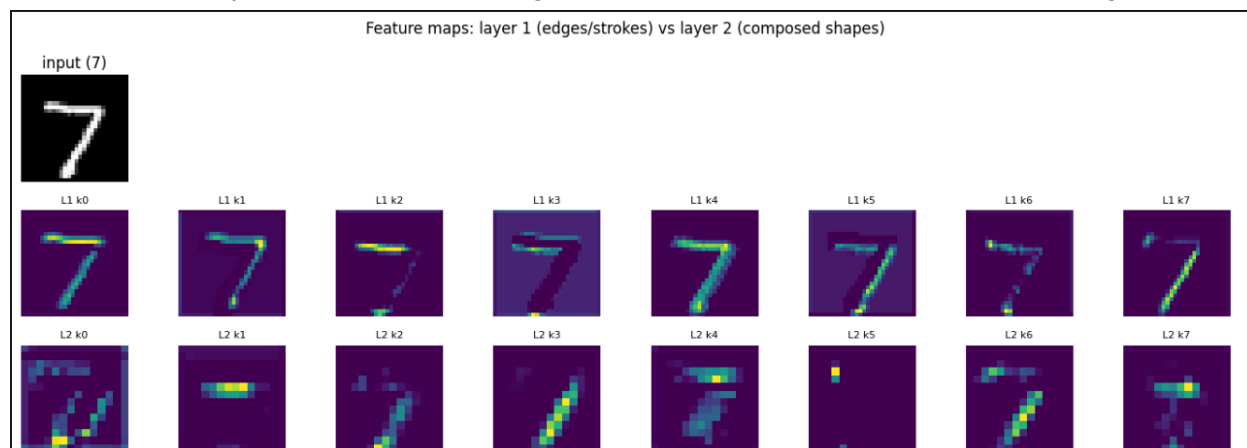
the whole thing is training it will get different versions of maybe the same picture making the training better since the ai has to learn stuff rather than just memorizing certain pixels. Now looking at the actual improvements it gave a pretty modest one which i would believe is cause the CNN in general already did reach 99% so its pretty hard to actually get better from there. It could also be that the mnist images are pretty much already normalised and centered so the training and testing are pretty much identical so nothing to really close the gap. And lastly i dont think augmentation really is that needed in my training image range or parameters id think augmentation really pays off when there isnt as much data to be used.

Now comparing FFN vs CNN well the ffn flattens the 28x28 before it starts with anything whilst the cnn has a 3x3 kernel that goes across the image now looking at my data it seems like my deeperCNN is better when it comes to lowering the loss i think adding the 2nd conv block basically is what made it better since it just made the ai learn better in a way that the first block would learn the edges and strokes and then the 2nd would just compose them to become more difficult shapes. Before going onwards let me talk about what the conv layer learned. From the gist of how i get it the conv kernel basically visualisation shows the learned 3x3 in the first conv layer which has the reconizable edges, strokes and so on. The feature map also makes the layer by layer abstraction so basically to put it short its low leveled features on the start whilst the higher features come later and it just all is built together by stacking these simple convolutions. But now to go further into it and show a bit of some pictures as well basically for the first picture you can see the first layer kernel, each tile basically shows the learned weights with the black pixels having a negative weight whilst white being the positive and grey being the middle ground.now as you can see they don't look like exactly like a textbook gradient edge detector but they are clearly directional contrast detectors . Most of the tiles that you can see have a split between the darker parts and lighter Which basically functions as a directional contrast detector



Now to go onto the feature map i basically sampled the number 7 and forwarded it through the network and tried to see what happens to it after each relu/conv block. You can see that the figure has 3 rows which row 1 shows the input, row 2 shows the first 8 feature maps from layer 1 and row 3 shows the first 8 feature maps from layer 2. (just to be clear these are just 8 of the first feature maps in reality there is 16 in layer 1 and 32 in layer 2) If you were to look closely you could see that layer 1 all of them clearly trace the 7 its basically still readable. Whilst when it comes to layer 2 its quite different it kinda just shows partial parts of the 7 like for example the

2nd one only shows a horizontal line whilst the 4th picture only has a diagonal line. This really shows how in layer 2 its started to complete certain parts of the 7 responding to different parts of it like a single stroke of the 7 rather than like layer 1 where it just has the whole 7 which i mean in the end is how CNN should be since layer 1 should be where the contrast and strokes are detected whilst layer 2 is there more to figure out the features and specialize in finding them;



So in the end I combine three different regularization techniques which pretty much went for different failure modes. First one i had was batchnorm2d which basically after an activations it normalized to have a zero mean and unit per batch what this helps with is just letting you use a higher learning rate. Amazingly enough the deepcnn reached around 98% testing accuracy after just one epoch which FFN took 5 epochs to get even close to. Then i also had dropout which basically after the first layer it ust randomly zeros 30% of the activations during training making the model have to use other representations in its many neurons instead of just relying on a few of them. Finally i had weight decay which added a L2 penalty on the weights by adam which just discourgaes large parameters.

So now for the hyperparameter tuning I basically just ran 4 different configs sweeps on the deepcnn which the first run won in its accuracy but then u could see in run 2 when the dropout was put in it was too much regularisation for the dataset. In run 3 it learned slowly, in run 4 with the 10x weight decay it became over regularised. Which from my understanding would mean that in this type of dataset like mnist the default of all of them like dropout=0.3 is better than pushing them past any of them it just hurts the dataset

Part 3:

So for part 3 let me begin by saying about what Data I used. So first of all i used the 10 class image dataset called 10animal from kaggle which i linked in my [readme.md](#) and at the beginning of my del3 notebook. So i loaded it all and split it by 70/15/15 for it to go into training/evaluation/testing whilst using the random split function with a seed just for it to be reproducable. All images were then resized and normalized so that it would work on both a model made from scratch and the pretrained model resnet50. In the end i the training set contained 18300 images.

When it came to the training i included augmentations like the flip, random affine and color jitter so that it wouldn't learn specific colors or pixels. So for the first model i made a classic CNN from scratch which had 4 conv blocks with everything they needed like the relu, flatten and so on which then trained for 15 epochs with adam. Which in the end it only reached 49% test accuracy with a loss of 1.4 which if you don't even have to look at it closely without seeing how poorly the results are. I'd believe the reasoning behind it would be a parameter to data mismatch since there's roughly around 12M parameters whilst the overall model is around 13M parameters which only got to train on 18k images from scratch without any pretraining. This model would have to learn edges, textures, eyes and everything that ResNet had already learnt all from a small dataset which in the end does show why it had such a bad accuracy when u check on it the train accuracy basically continued to climb up whilst the validation just plateaued which basically means that it was underfit conv or a overfit fc.

So now what about transfer learning? Well for transfer learning i loaded in as i said before resnet50 which has pretrained weights on imagenet. I replaced only really the final layer where it actually classifies so it outputs 10 logits instead of its 1000 then used adam to get it to a small learning rate. So the pretrained features are just really adjusted by a bit rather than fully replacing/overwriting them and after only 8 epochs it reached around 97% test accuracy and its validation curve crossed 90% in the first epoch which shows how well it did with its pretrained weights. Now comparing the 2 you don't even need to glance at it for long to know which one came out ahead and is better. The transfer learning model achieved basically 2x the accuracy of the from scratch CNN even with it having less epochs to train with.

How i like to think of it is that making a new model from scratch is like teaching a baby what a car is or what a plant is, how to know if it is a plant or a car, or if its a dog or a cat and so on. The baby would have to learn from scratch like oh it has ears, it has certain features and sometimes it might mistake the two but with more data they would understand it better.

Now to get into why it actually just works better to use transfer learning. So earlier in my report i talked about how some layers learn different features and the more later into the layer you get the more features it detects until it sees stuff fully like a horizontal line of a 7. Now with these pictures of animals it has a lot more to learn which in the early stages of learning it has to do all of that, learn the edges, gradients, colours, eyes, fur, paws and so on now using resnet which had already learnt all of that we could just reuse everything it has learnt and use it on our own animal pictures since animals basically share the same visual structure as the 1000 imagenet classes. By me only replacing the final layer and changing it to have a smaller learning rate i basically just effectively reused the entire features and only had to make it learn the mapping from those features to the different animals i have in my data. Whilst the from scratch had to learn everything from just 18k images which is waaay too little. Imagenet itself used 1.2 millions images for the same thing. Just a little point to make because for transfer learning to work i need to input pretrained weights that basically expect the same inputs in the exact same distribution as my mean and std list did.

And finally for this part I'd like to talk about the ResNet 1x1 kernels. So the special thing with ResNet50 is that its bottleneck blocks pretty much contain three convolutions which go 1x1 to 3x3 to 1x1. The 1x1 convolutions don't really see the other parts; they just mix channels at single pixels. How the flow goes is that the first conv reduces the channel count before the more expensive part of the conv starts, then it expands back to its original. At the end, all of it together makes the depth of ResNet computationally tractable and keeps the parameters at a reasonable amount.

To conclude, I'd say the biggest takeaway from all of this is that the difficulty curve flattens once you stop going against the fundamentals. Through part 1, I learned the math parts of it but nothing really much other than that. In part 2, I learned a lot more about PyTorch and a little more of how AI's are created and then finally. In part 3, it shows that the right move is not to train your own network but just reusing someone else's which was trained on a much larger dataset.